POINTEURS & MÉMOIRE

1. Définition :

Un **pointeur** est une **variable qui stocke l'adresse mémoire d'une autre variable**.

- Chaque variable en mémoire possède une **adresse unique**, et un pointeur peut "pointer" vers cette adresse.

2. Explication:

- Le type du pointeur doit **correspondre au type de la variable pointée** (ex : int* pour un int).
- Les pointeurs permettent :
  - Manipuler directement la mémoire.
  - Passer des variables par référence aux fonctions.
  - Créer des tableaux dynamiques ou des structures complexes.
- **Opérateurs importants :**
  - `&` → adresse d'une variable.
  - `*` → valeur pointée par le pointeur (déréférencement).

3. Syntaxe de base :

- **Déclaration d’un pointeur:**

```
type *nom_pointeur;
```

- **Initialisation :**

```
int a = 10;
int *p = &a; // p pointe vers a
```

- **Accès à la valeur :**

```
printf("%d\n", *p); // affiche 10
```

4. Exemple simple :

```
#include <stdio.h>

int main() {

   int a = 20;

   int *p = &a; // pointeur vers a

   printf("Adresse de a: %p\n", p);   // affiche l'adresse de a

   printf("Valeur de a: %d\n", *p);   // affiche la valeur de a

   *p = 50; // modification via le pointeur

   printf("Nouvelle valeur de a: %d\n", a);

   return 0;

}
```

5. Pointeurs et tableaux:

- Un tableau est **un pointeur vers son premier élément**.
- On peut accéder aux éléments par pointeur ou par indice.

```
#include <stdio.h>
```

```
int main() {
    int tab[3] = {10, 20, 30};
    int *p = tab; // pointeur vers le premier élément
    for(int i = 0; i < 3; i++) {
        printf("tab[%d] = %d\n", i, *(p + i)); // équivalent à tab[i]
    }
    return 0;
}
```

6. Pointeurs et fonctions:

- Permet de **modifier une variable passée en paramètre**.
- Utile pour **passer des tableaux ou des grandes structures** sans copier la mémoire.

```
#include <stdio.h>
void incrementer(int *x) {
    *x = *x + 1; // modification de la valeur pointée
}
int main() {
    int a = 10;
    incrementer(&a);
    printf("Nouvelle valeur de a: %d\n", a); // 11
    return 0;
}
```

7. Pointeurs et allocation dynamique (malloc et free):

- Permet de créer des **tableaux dynamiques ou structures** dont la taille est connue **à l'exécution**.
- Libérer la mémoire avec free() pour éviter les fuites.

```
#include <stdio.h>

#include <stdlib.h>

int main() {

   int n = 5;

   int *tab = (int*) malloc(n * sizeof(int)); // allocation dynamique

   for(int i = 0; i < n; i++) {

      tab[i] = i + 1;

   }

   for(int i = 0; i < n; i++) {

      printf("%d ", tab[i]);

   }

   free(tab); // libérer la mémoire

   return 0;

}
```

8. Pointeurs et matrices dynamiques:

- Crée une **matrice dynamique** avec pointeurs de pointeurs.

```
#include <stdio.h>

#include <stdlib.h>

int main() {
```

```
    int lignes = 2, colonnes = 3;
    int **matrice = (int**) malloc(lignes * sizeof(int*));
    for(int i = 0; i < lignes; i++)
        matrice[i] = (int*) malloc(colonnes * sizeof(int));
    // remplir la matrice
    int compteur = 1;
    for(int i = 0; i < lignes; i++)
        for(int j = 0; j < colonnes; j++)
            matrice[i][j] = compteur++;
    // afficher
    for(int i = 0; i < lignes; i++){
        for(int j = 0; j < colonnes; j++)
            printf("%d ", matrice[i][j]);
        printf("\n");
    }
    // libérer la mémoire
    for(int i = 0; i < lignes; i++)
        free(matrice[i]);
    free(matrice);
    return 0;
}
```